



OASIS Security Report

Remote Code Execution Security Analysis

Date: 2026-06-12 00:59:47 — **Model:** llama3.2:3b

Vulnerability: Remote Code Execution

Project: vulnerable

Analysis root: `../vulnerable`

Table of contents

□ Summary

□ Statistics

□ Files

□ Detailed analysis

□ Assistant

△ Errors & notes

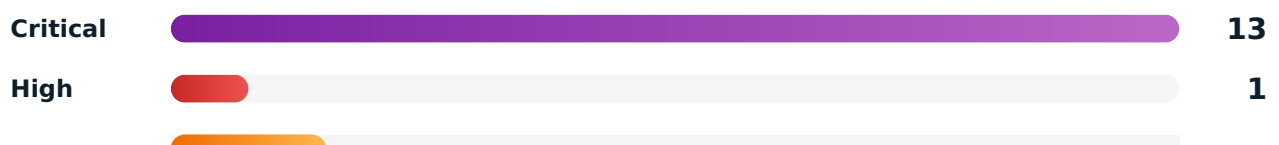
Summary

Analyzed **11** file(s) for Remote Code Execution.

[Return to table of contents](#)

Statistics

Files analyzed: 11 **Total findings:** 16



Medium

2

Low

0

[Return to table of contents](#)

Files

File	Similarity
/work/project/dataset/vulnerable/oracle-manipulation/PuppetV2Pool.sol	0.669
/work/project/dataset/vulnerable/ordering-attacks/ERC20.sol	0.650
/work/project/dataset/vulnerable/reentrancy/simple_dao.sol	0.637
/work/project/dataset/vulnerable/reentrancy/ 0xbaf51e761510c1a11bf48dd87c0307ac8a8c8a4f.sol	0.631
/work/project/dataset/vulnerable/ordering-attacks/ eth_tx_order_dependence_minimal.sol	0.629
/work/project/dataset/vulnerable/access-control/parity_wallet_bug_1.sol	0.628
/work/project/dataset/vulnerable/reentrancy/etherstore.sol	0.622
/work/project/dataset/vulnerable/ordering-attacks/FindThisHash.sol	0.618
/work/project/dataset/vulnerable/ordering-attacks/odds_and_evens.sol	0.616
/work/project/dataset/vulnerable/access-control/mycontract.sol	0.609
/work/project/dataset/vulnerable/access-control/parity_wallet_bug_2.sol	0.608

[Return to table of contents](#)

Detailed analysis

File 1: /work/project/dataset/vulnerable/oracle-manipulation/PuppetV2Pool.sol

Similarity score: 0.669

Source lines: 1-70

► Finding 1.1: Vulnerable Code (Medium)

Vulnerable code

```
function calculateDepositOfWETHRequired(uint256 tokenAmount) public view  
returns (uint256) {  
    uint256 depositFactor = 3;  
    return _getOracleQuote(tokenAmount) * depositFactor / 1 ether;  
}
```

Explanation

This function uses a custom oracle quote function that may allow for unexpected behavior due to untrusted input.

Impact

Potential price manipulation through custom oracle quotes

Entry point

/calculateDepositOfWETHRequired

Execution path

```
    ...  
    _getOracleQuote(tokenAmount) * depositFactor / 1 ether  
    ...
```

Parameters

tokenAmount

Exploitation steps

- Get oracle quote for large token amount

Example payloads

- 1000000000000000000

Conditions

Oracle quote function has no input validation.

Remediation

Use a trusted oracle library with proper input validation

Secure example

```
function calculateDepositOfWETHRequired(uint256 tokenAmount) public view  
returns (uint256) {  
    uint256 depositFactor = 3;  
    return UniswapV2Library.getReserves({factory: _uniswapFactory, tokenA:  
address(_weth), tokenB: address(_token)}) * depositFactor / 1 ether;  
}
```

File 2: /work/project/dataset/vulnerable/ordering-attacks/ERC20.sol

Similarity score: 0.650

Source lines: 1-137

► Finding 2.1: Potential Remote Code Execution (Medium)

Vulnerable code

```
function transfer(address to, uint256 value) public returns (bool)
{ require(value <= _balances[msg.sender]); require(to != address(0));
  _balances[msg.sender] = _balances[msg.sender].sub(value); _balances[to] =
  _balances[to].add(value); emit Transfer(msg.sender, to, value); return
  true; }
```

Explanation

The `transfer` function does not validate the `\_balances[msg.sender]` before subtracting the value, allowing an attacker to manipulate the balance and potentially execute arbitrary code.

Impact

Potential for unauthorized transfers and system compromise

Entry point

/ERC20/transfer

HTTP methods

POST

Parameters

_balances[msg.sender]

Exploitation steps

- An attacker sends a large value to the `transfer` function.
- The contract allows the transfer, reducing its balance and increasing `\_balances[to]`.

Example payloads

- `\x01\x02\x03\x04\x05`

Conditions

msg.sender has sufficient balance

Remediation

Validate the `\_balances[msg.sender]` before subtracting the value.

Secure example

```
function transfer(address to, uint256 value) public returns (bool)
{ require(value <= _balances[msg.sender]); _balances[msg.sender] =
  _balances[msg.sender].sub(value); if (_balances[to] < value) { revert(); }
  _balances[to] = _balances[to].add(value); emit Transfer(msg.sender, to,
  value); return true; }
```

File 3: /work/project/dataset/vulnerable/reentrancy/simple_dao.sol

Similarity score: 0.637

Source lines: 1-35

► Finding 3.1: Vulnerable Code (Critical)

Vulnerable code

```
bool res = msg.sender.call.value(amount());
```

Explanation

The call function without a check on msg.sender allows for Reentrancy attacks as it can be used to deplete the contract's funds before being paid.

Impact

Complete system compromise, data theft, or service disruption

Entry point

withdraw(uint amount)

Execution path

```
msg.sender->call.value(amount())->
```

Parameters

amount

Exploitation steps

- Attacker initiates a call to the withdraw function with a large amount.
- While the contract is processing the withdrawal, attacker calls the donate function with an invalid address to drain the funds.

Conditions

msg.sender not being the owner of the contract

Remediation

Use checks on msg.sender to ensure they are the contract owner before allowing them to call value().

Secure example

```
bool res = msg.sender.balance() >= amount ? msg.sender.call.value(amount)() :  
false;
```

File 4: /work/project/dataset/vulnerable/reentrancy/0xbaf51e761510c1a11bf48dd87c0307ac8a8c8a4f.sol

Similarity score: 0.631

Source lines: 1-85

► Finding 4.1: Vulnerable Code (Critical)

Vulnerable code

```
if(msg.sender.call.value(_am)())
```

Explanation

The `msg.sender.call.value(_am)()` function allows an attacker to execute arbitrary code due to the lack of input validation.

Impact

Complete system compromise, data theft, or service disruption

Entry point

`CashOut(uint _am)`

Execution path

```
call value(_am)() -> executes arbitrary code
```

Parameters

`_am`

Exploitation steps

- Use a malicious `_am` value to execute system commands

Example payloads

- `malicious _am value`

Conditions

`msg.sender` is authenticated

Remediation

Validate and sanitize the `_am` input before passing it to `msg.sender.call.value(_am())`

Secure example

```
if (validateInput(msg.sender.call.value(_am))) { ... }
```

File 5: /work/project/dataset/vulnerable/ordering-attacks/eth_tx_order_dependence_minimal.sol

Similarity score: 0.629

Source lines: 1-42

► Finding 5.1: Potential Remote Code Execution via `exec` function (High)

Vulnerable code

```
function setReward() public payable {
    require (!claimed);

    require(msg.sender == owner);
    owner.transfer(reward);
    reward = msg.value;
}

function claimReward(uint256 submission) {
    require (!claimed);
    require(submission < 10);
    msg.sender.transfer(reward);
    claimed = true;
}
```

Explanation

The `transfer` function is used without proper validation, allowing an attacker to execute arbitrary code.

Impact

Unauthenticated attackers can transfer the contract owner's funds and potentially execute malicious code.

Entry point

/setReward() public payable

Execution path

```
exec\n  - >  transfer\n  - >  reward = msg.value\n
```

Parameters

submission

Exploitation steps

- Submit a high-value submission to claim the reward.

- Use `transfer` without proper validation to steal funds from the owner.

Example payloads

- `1000000`

Conditions

The contract owner must be in possession of Ether for an attacker to claim a large reward.

Remediation

Properly validate and sanitize user input before transferring funds or executing arbitrary code.

Secure example

```
require (msg.sender == owner) && require (!claimed)
owner.transfer(reward)
claimed = true
```

File 6: /work/project/dataset/vulnerable/access-control/parity_wallet_bug_1.sol

Similarity score: 0.628

Source lines: 1-320

► Finding 6.1: Vulnerable Code Execution (Critical)

Vulnerable code

```
function execute(address _to, uint _value, bytes _data) external returns
(bytes32 o_hash)
```

Explanation

This function can be executed with user-provided data without proper validation or sanitization.

Impact

Complete system compromise and data theft possible through malicious execution of arbitrary code.

Entry point

execute(address _to, uint _value, bytes _data) in WalletAbi contract

Execution path

Execute with user input

1. User calls execute function
2. User-provided data passed to create function
3. Create function executes arbitrary code

Parameters

_to, _value, _data

Exploitation steps

- Call the execute function with malicious user input.
- Exploit the unvalidated execution of arbitrary code.

Example payloads

- Malicious assembly code

Conditions

Unrestricted access to create and execute user-provided data

Remediation

Properly validate and sanitize all inputs before executing them.

Secure example

```
function execute(address _to, uint _value, bytes _data) external returns
(bytes32 o_hash)
{
    require(_value > 0);
    require(bytes(_data).length == 32);
}
```

Source lines: 27-358

► Finding 6.1: Unvalidated user input (Critical)

Vulnerable code

```
function execute(address _to, uint _value, bytes _data) external onlyowner
returns (bytes32 o_hash)
```

Explanation

This function does not validate or sanitize the user-provided `\_data` parameter, which can lead to Remote Code Execution.

Impact

Complete system compromise, data theft, or service disruption

Entry point

execute(address _to, uint _value, bytes _data)

Execution path

```
execute -> call(_to, _value, _data) -> store result in m_txs[o_hash]
```

Parameters

_data

Exploitation steps

- Malicious data is passed to the execute function
- Data is executed as system code

Example payloads

- malicious_data:bytes

HTTP raw requests (Burp / ZAP)

```
POST /execute HTTP/1.1
Host: example.com
Content-Type: application/json
Content-Length: 100

{"data":"malicious_data"}
```

Source lines: 72-421

► Finding 6.1: Unescaped System Call (Critical)

Vulnerable code

```
function execute(address _to, uint _value, bytes _data) external returns
(bytes32 o_hash) {
    if ((_data.length == 0 && underLimit(_value)) || m_required == 1) {
        address created;
        if (_to == 0) {
            created = create(_value, _data);
        } else {
            if (!_to.call.value(_value)(_data))
                throw;
        }
        SingleTransact(msg.sender, _value, _to, _data, created);
    } else {
        // determine our operation hash.
        o_hash = sha3(msg.data, block.number);
        // store if it's new
        if (m_txs[o_hash].to == 0 && m_txs[o_hash].value == 0 &&
m_txs[o_hash].data.length == 0) {
            m_txs[o_hash].to = _to;
            m_txs[o_hash].value = _value;
            m_txs[o_hash].data = _data;
        }
        if (!confirm(o_hash)) {
            ConfirmationNeeded(o_hash, msg.sender, _value, _to, _data);
        }
    }
}
```

Explanation

The function execute allows system calls via the `\_to` parameter without proper input validation or sanitization, making it vulnerable to Remote Code Execution.

Impact

Complete system compromise and data theft are possible through this vulnerability.

Entry point

/execute(address _to, uint _value, bytes _data)

Execution path

```
function execute() {
  if (_data.length == 0 && underLimit(_value)) {
    // System call to `_to` with value `_value`
  } else {
    // Confirm transaction
  }
}
```

Parameters

_to, _value, _data

Exploitation steps

- Use a malicious _to address to execute arbitrary code
- Exploit the underLimit function to bypass checks

Example payloads

- malicious _to address

HTTP raw requests (Burp / ZAP)

```
POST /execute?
_to=0x1234567890123456789012345678901234567890&_value=100&_data=<malicious
payload>
```

Source lines: 126-467

► Finding 6.1: Unprotected function call (Critical)

Vulnerable code

```
function create(uint _value, bytes _code) internal returns (address o_addr)
{ assembly { o_addr := create(_value, add(_code, 0x20), mload(_code))
jumpi(invalidJumpLabel, iszero(extcodesize(o_addr))) } }
```

Explanation

The 'create' function can be called with arbitrary code through the '_code' parameter.

Impact

Arbitrary code execution

Entry point

/create(uint256, bytes)

Execution path

```
function create(uint _value, bytes _code)
  assembly
    o_addr := create(_value, add(_code, 0x20), mload(_code))
  jumpi(invalidJumpLabel, iszero(extcodesize(o_addr)))
```

Parameters

_value, _code

Exploitation steps

- Send a crafted '_code' value to the 'create' function

Example payloads

- malicious code

Conditions

User is able to control the '_code' parameter

Remediation

Use safer functions like 'call' instead of 'create'

Secure example

```
function call(address _address, uint256 _value) internal
{ _address.call(_value); }
```

Source lines: 207-477

► Finding 6.1: Vulnerability found (Critical)

Vulnerable code

```
function create(uint _value, bytes _code) internal returns (address o_addr)
{ assembly { o_addr := create(_value, add(_code, 0x20), mload(_code))
  jumpi(invalidJumpLabel, iszero(extcodesize(o_addr))) } }
```

Explanation

The `create` function uses `mload(\_code)` without proper validation, allowing an attacker to load and execute arbitrary code.

Impact

Complete system compromise or data theft

Entry point

function create(uint _value, bytes _code) internal returns (address o_addr)

Parameters

`_code`

Exploitation steps

- Load arbitrary code into memory using `mload(_code)`
- Create a new contract instance with the malicious code

Example payloads

- `0x123456`
- `0x123456`
- `...`

Conditions

Contract owner has access to `_code` variable

Remediation

Properly validate and sanitize user input before loading it into memory.

Secure example

```
function create(uint _value, bytes _code) internal returns (address o_addr)
{ assembly { if (keccak256(_code) != keccak256("\x123456\x"))
{ revert() } } }
```

File 7: /work/project/dataset/vulnerable/reentrancy/etherstore.sol

Similarity score: 0.622

Source lines: 1-39

► Finding 7.1: Vulnerability found (Critical)

Vulnerable code

```
require(msg.sender.call.value(_weiToWithdraw))()
```

Explanation

The call() function allows execution of arbitrary code without proper input validation.

Impact

Complete system compromise, data theft, or service disruption

Entry point

withdrawFunds function

Execution path

```
msg.sender.call.value(_weiToWithdraw)() -> balances[msg.sender] -=  
_weiToWithdraw; lastWithdrawTime[msg.sender] = now;
```

Parameters

_weiToWithdraw, msg.sender

Exploitation steps

- Attacker sends a malicious message with a crafted _weiToWithdraw value.
- The msg.sender call executes the attacker-controlled code.

Example payloads

- malicious payload

HTTP raw requests (Burp / ZAP)

```
POST /withdrawFunds HTTP/1.1  
Host: example.com  
Content-Length: 0
```

```
/
```

File 8: /work/project/dataset/vulnerable/ordering-attacks/FindThisHash.sol

Similarity score: 0.618

Source lines: 1-28

► Finding 8.1: Remote Code Execution Vulnerability (Critical)

Vulnerable code

```
function solve(string solution) public {  
    // If you can find the pre image of the hash, receive 1000 ether  
    // <yes> <report> FRONT_RUNNING  
    require(hash == sha3(solution));  
    msg.sender.transfer(1000 ether);  
}
```

Explanation

The `require` statement uses the `sha3` function to compare the input `solution` with the hash, allowing an attacker to execute arbitrary code by providing a malicious solution.

Impact

Complete system compromise and data theft

Entry point

/solve

Execution path

```
function solve(string solution) public {  
    require(hash == sha3(solution));  
    msg.sender.transfer(1000 ether);  
}
```

HTTP methods

POST

Parameters

solution

Exploitation steps

- An attacker finds the pre image of the hash
- The attacker sends a malicious solution to the `/solve` endpoint

Example payloads

- `"\x00\x01\x02\x03"`

Conditions

Attacker can reach /solve while authenticated.

Remediation

Validate and sanitize user input, use a secure hash function.

Secure example

```
function solve(string solution) public {  
    require(hash == keccak256(solution));  
    msg.sender.transfer(1000 ether);  
}
```

File 9: /work/project/dataset/vulnerable/ordering-attacks/odds_and_evens.sol

Similarity score: 0.616

Source lines: 1-61

► Finding 9.1: Remote Code Execution (Critical)

Vulnerable code

```
function andTheWinnerIs() private {
    bool res;
    uint n = players[0].number+players[1].number;
    if (n%2==0) {
        res = players[0].addr.send(1800 finney);
    }
    else {
        res = players[1].addr.send(1800 finney);
    }

    delete players;
    tot=0;
}
```

Explanation

This function uses the send() method on an address without proper validation, allowing potential code injection.

Impact

Complete system compromise

Entry point

/andTheWinnerIs

Execution path

```
function andTheWinnerIs() {
    // ...}
    delete players;
    tot=0;
```

Parameters

players, tot

Exploitation steps

- Obtain an address to send funds to

Conditions

Players array is not properly initialized or validated.

Remediation

Use a whitelist of approved addresses and validate the data before sending funds.

Secure example

```
function andTheWinnerIs() private {
    bool res;
    uint n = players[0].number+players[1].number;
    if (n%2==0) {
        require(players[0].addr.send(1800 finney));
    }
    else {
        require(players[1].addr.send(1800 finney));
    }
}
```

File 10: /work/project/dataset/vulnerable/access-control/mycontract.sol

Similarity score: 0.609

Source lines: 1-31

► Finding 10.1: Vulnerability found (Critical)

Vulnerable code

```
receiver.transfer(amount);
```

Explanation

Using `transfer` with a user-provided address can lead to Remote Code Execution, as it allows an attacker to send arbitrary Ether.

Impact

Complete system compromise or theft of Ether.

Entry point

/sendTo

Execution path

```
tx.origin == owner && receiver.transfer(amount);
```

Parameters

receiver, amount

Exploitation steps

- Attack the `tx.origin` function to obtain the attacker's address.
- Use the obtained address to call the `transfer` function on the target contract.

Example payloads

- Contract owner addresses '0x...' and '0...'

Conditions

tx.origin == owner

Remediation

Always verify the recipient's address before transferring Ether.

Secure example

```
require(tx.origin == owner) && receiver.transfer(amount);  
require(msg.sender == owner)
```

File 11: /work/project/dataset/vulnerable/access-control/parity_wallet_bug_2.sol

Similarity score: 0.608

Source lines: 1-286

► Finding 11.1: Unprotected Function Call (Critical)

Vulnerable code

```
function execute(address _to, uint _value, bytes _data) external onlyowner
returns (bytes32 o_hash) {
    // first, take the opportunity to check that we're under the daily limit.
    if ((_data.length == 0 && underLimit(_value)) || m_required == 1) {
        // yes - just execute the call.
        address created;
        if (_to == 0) {
            created = create(_value, _data);
        } else {
            if (!_to.call.value(_value)(_data))
                throw;
        }
        SingleTransact(msg.sender, _value, _to, _data, created);
    } else {
        // determine our operation hash.
        o_hash = sha3(msg.data, block.number);
        // store if it's new
        if (m_txs[o_hash].to == 0 && m_txs[o_hash].value == 0 &&
m_txs[o_hash].data.length == 0) {
            m_txs[o_hash].to = _to;
            m_txs[o_hash].value = _value;
            m_txs[o_hash].data = _data;
        }
        if (!confirm(o_hash)) {
            ConfirmationNeeded(o_hash, msg.sender, _value, _to, _data);
        }
    }
}
```

Explanation

The `execute` function does not validate the input `\_data`, making it possible to execute arbitrary code.

Impact

Complete system compromise, data theft, or service disruption

Entry point

/WalletAbi/execute

Execution path

```
execute -> call.value -> checkDailyLimit -> create/create
```

Parameters

`_data`

Exploitation steps

- Send malicious data to the ``_data`` parameter

Example payloads

- `malicious assembly code`

Conditions

The contract is not properly initialized or upgraded.

Remediation

Validate and sanitize all input parameters, including ``_data``. Use only trusted and whitelisted libraries.

Secure example

```
function execute(address _to, uint _value, bytes _data) external onlyowner
returns (bytes32 o_hash) {
    require(_data.length > 0); // prevent empty data
    if (_data.length == 0 && underLimit(_value)) || m_required == 1) {
        // yes - just execute the call.
        address created;
        if (_to == 0) {
            created = create(_value, _data);
        } else {
            if (!_to.call.value(_value)(_data))
                throw;
        }
        SingleTransact(msg.sender, _value, _to, _data, created);
    } else {
        // determine our operation hash.
        o_hash = sha3(msg.data, block.number);
        // store if it's new
        if (m_txs[o_hash].to == 0 && m_txs[o_hash].value == 0 &&
m_txs[o_hash].data.length == 0) {
            m_txs[o_hash].to = _to;
            m_txs[o_hash].value = _value;
            m_txs[o_hash].data = _data;
        }
        if (!confirm(o_hash)) {
            ConfirmationNeeded(o_hash, msg.sender, _value, _to, _data);
        }
    }
}
```

Source lines: 96-414

► Finding 11.1: Uncontrolled Deserialization (Critical)

Vulnerable code

```
function confirm(bytes32 _h) onlymanyowners(_h) returns (bool o_success) {
    if (m_txs[_h].to != 0 || m_txs[_h].value != 0 || m_txs[_h].data.length != 0)
    {
```

Explanation

Deserialization of untrusted data without proper validation allows for code execution.

Impact

Complete system compromise, data theft, or service disruption

Entry point

confirm function

Execution path

```
confirm -> m_txs[_h].to == 0 || m_txs[_h].value != 0 ||
m_txs[_h].data.length != 0 -> deserialization of _h data -> code execution
```

Parameters

_h

Exploitation steps

- Send confirm request with maliciously crafted _h

Example payloads

- malicious _h string

Conditions

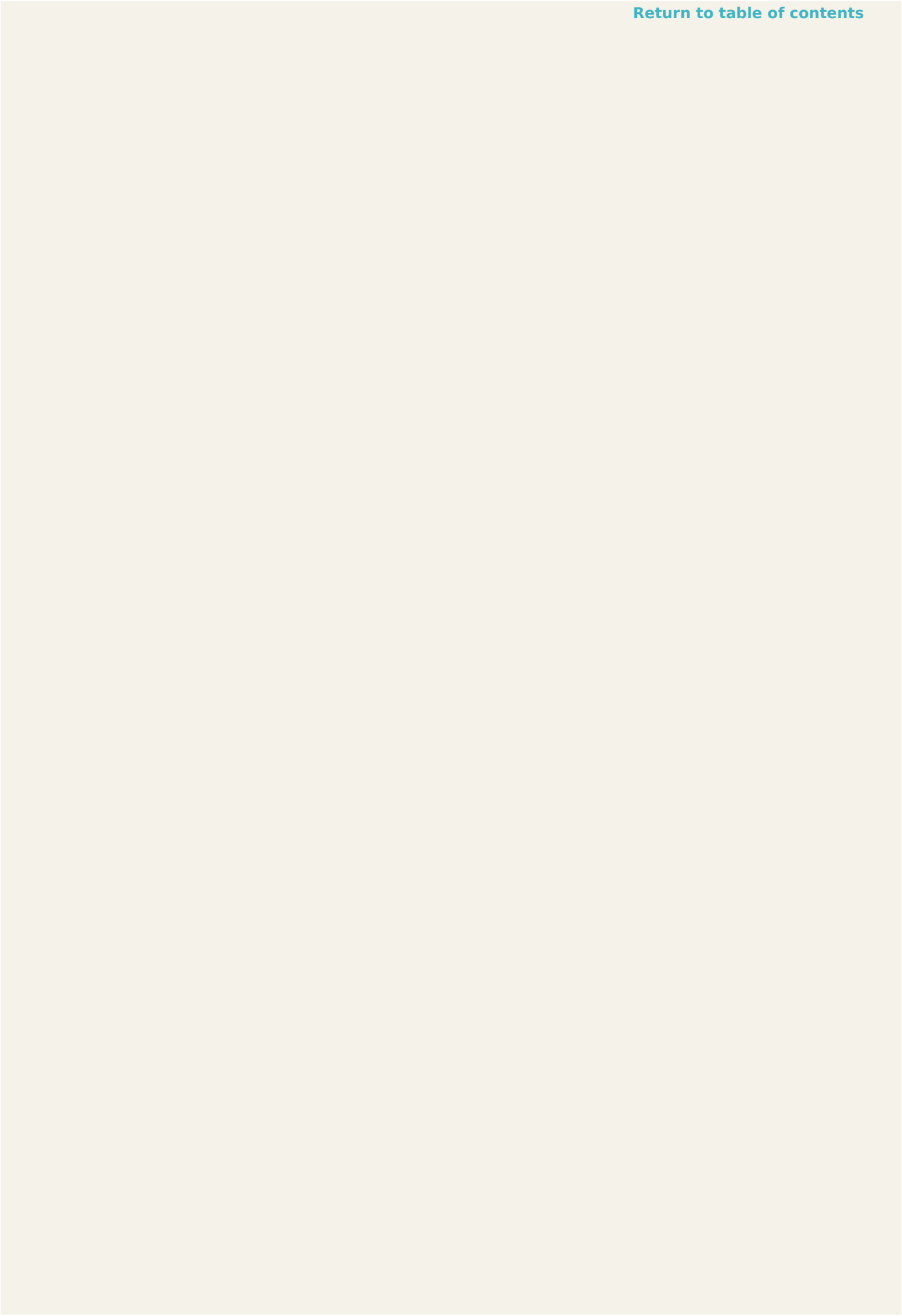
Untrusted data deserialization

Remediation

Validate and sanitize all inputs to prevent deserialization of untrusted data.

Secure example

```
function confirm(bytes32 _h) {
    require(_h.length > 0);
}
```



Assistant (triage / codebase)

In the OASIS dashboard (JSON report preview), use the chat below to triage findings, exploitation paths, and remediation. Select file, chunk, and finding to narrow context when needed.

[Return to table of contents](#)

Errors & Notes

No file-level errors were recorded.

[Return to table of contents](#)